



Livrable E2

Veille technique et réglementaire



Franck Le Fur

Le 30 Mars 2026

Titre professionnel **Développeur en intelligence artificielle**
de niveau 6 enregistré au RNCP sous le n°37827.

Passage par la voie de la formation - parcours de 19 mois achevé en mars 2026

Table des matières

1	Présentation	2
2	Organisation de la veille technologique [C6]	3
2.1	Les sources	3
2.2	Automatiser la collecte des sources	3
2.2.1	L'API Arxiv	3
2.2.2	Les alertes Google Scholar	4
3	Représentations Discrètes et Fréquentielles [C6]	5
3.1	Encodage One-Hot	5
3.2	Le Modèle Bag-of-Words (BoW)	5
3.3	TF-IDF (Term Frequency-Inverse Document Frequency)	7
3.4	Limitations de ces méthodes	7
4	Les Plongements Lexicaux Statiques et Prédicatifs (Word Embeddings) [C6]	8
4.1	Word2Vec : Architectures Neuronales Légères	8
4.2	GloVe (Global Vector)	9
4.3	FastText : Robustesse par les Sous-Mots (Subword)	10
4.4	La Limite de la Sémantique Statique	11
5	L'Encodage Contextuel et des Transformers [C6]	12
5.1	L'Architecture Transformer : le mécanisme d'Attention	12
5.2	BERT (Bidirectional Encoder Representations from Transformers)	12
5.3	GPT (Generative Pre-trained Transformer)	13
5.4	Choix du modèle : Encodage vs. Génération	13
6	Services d'intelligence artificielle pour de l'embedding [C7, C8]	14
6.1	Mistral	14
6.2	Cohere	14
6.3	Hugging Face	15
6.4	OpenRouter	16
7	Évaluation des modèles d'embeddings [C7]	17
7.1	Évaluation à l'aide de MTEB	17
7.1.1	Les 8 Catégories de Tâches de MTEB	17
7.1.2	Évaluation des modèles sur la tâche de similarité sémantique	18
7.2	Leaderboard MTEB	21
8	Conclusion	22
8.1	Tendances Futures	22

1 Présentation

Mon projet de société au **crédit Agricole** consistait en la création d'un moteur de recherche sur l'ensemble des documents d'Assurances, l'efficacité d'un tel moteur repose sur les performances de la recherche de **similarité sémantique** c'est-à-dire sur la capacité à notre moteur à comprendre **la syntaxe du langage** (loi qui régit l'agencement des structures des mots dans les phrases) et à **capturer leur signification et leurs relations sémantiques**, c'est ainsi que la question de la **vectorisation des textes (ou encodage)** s'est posée.

Les ordinateurs travaillent exclusivement avec des 0 et des 1, si la conversion des nombres vers une représentation binaire est naturelle, la consersion des données non-structurées en nombres est plus compliquée. Comment alors travailler avec des images ou du texte ?

La conversion d'images en nombres se fait assez naturellement si l'on considère une image comme un tableau de pixels et chaque pixel comme un vecteur numérique correspondant à la couleur RGB, la transparence etc ...

La vectorisation du texte est moins naturelle et il existe un grand nombre de techniques.

Dans cette veille nous ferons un état des lieux des méthodes de vectorisation existantes, des méthodes basiques d'**encodage discret ou fréquentiel**, des **plongements lexicaux** et des méthodes basées sur l'**encodage contextuel avec les transformers**. Nous évaluerons certains modèles de vectorisation sur les Dataset classiques **MTEB** conçus pour différentes tâches de traitement du langage naturel et en particulier celui qui nous intéresse : **La similarité sémantique** et nous comparerons les modèles dans les Leaderboard de sites de références afin de choisir le modèle le plus adapté à notre projet.

Cette veille technique sera exclusivement mené à l'aide du langage python, Le dépôt **github** du projet est consultable ici : https://github.com/Franck-LF/Veille_Technique

2 Organisation de la veille technologique [C6]

2.1 Les sources

Pour réaliser cette veille technique sur les méthodes de vectorisation de documents, nous consultons des sites tels que : GeeksForGeeks, Archiv.org, Google Scholar.

GeeksForGeeks est utilisé pour collecter des informations sur les méthodes "anciennes", par exemple les méthodes basées sur la fréquence des mots comme **Bag-Of-Words**, **TF-IDF** ou les méthodes basées sur le plongement lexical (**GLOVE**, **Word2Vec**), Au contraire les sources comme Archiv.org et Google Scholar sont utilisées pour les méthodes plus récentes (transformers) et pour se maintenir à jour des dernières évolutions.

2.2 Automatiser la collecte des sources

2.2.1 L'API Arxiv

Le site **Arxiv.org** offre une API, laquelle peut être requêter à l'aide d'un module python,

```
1 import arxiv
2
3 search = arxiv.Search(
4     query="embedding",
5     max_results=5,
6     sort_by=arxiv.SortCriterion.SubmittedDate
7 )
8
9 for paper in search.results():
10     print(f"Titre : {paper.title}")
11     print(f"Auteurs : {paper.authors}")
12     print(f"Resume : {paper.summary[:200]}...")
13     print(f"Lien : {paper.pdf_url}\n")
```

Titre : STEM: Scaling Transformers with Embedding Modules

Auteurs : [arxiv.Result.Author('Ranajoy Sadhukhan'), arxiv.Result.Author('Sheng Cao'), arxiv.Result.Author('Harshad Kulkarni')]

Résumé : Fine-grained sparsity promises higher parametric capacity without proportional per-token compute, but current methods are limited by the need for a large number of parameters.

Lien : <https://arxiv.org/pdf/2601.10639v1>

Titre : Circumplanetary Disk Candidate in the Disk of HD 163296 Traced by Localized Emission from Simple Organic Molecules

Auteurs : [arxiv.Result.Author('Andres F. Izquierdo'), arxiv.Result.Author('Jaehan Bae'), arxiv.Result.Author('M. J. Barlow')]

Résumé : Atacama Large Millimeter/submillimeter Array observations suggest that the disc of HD 163296 is being eroded by the stellar wind.

Lien : <https://arxiv.org/pdf/2601.10631v1>

Titre : RSATalker: Realistic Socially-Aware Talking Head Generation for Multi-Turn Conversation

Auteurs : [arxiv.Result.Author('Peng Chen'), arxiv.Result.Author('Xiaobao Wei'), arxiv.Result.Author('Yi Yang')]

Résumé : Talking head generation is increasingly important in virtual reality (VR), especially for social scenarios. Existing methods often fail to generate realistic and socially-aware talking heads.

Lien : <https://arxiv.org/pdf/2601.10606v1>

Titre : Institutional AI: A Governance Framework for Distributional AGI Safety

Auteurs : [arxiv.Result.Author('Federico Pierucci'), arxiv.Result.Author('Marcello Galisai'), arxiv.Result.Author('Giovanni De Luca')]

Résumé : As LLM-based systems increasingly operate as agents embedded within human social and technical systems, the need for robust governance frameworks becomes critical.

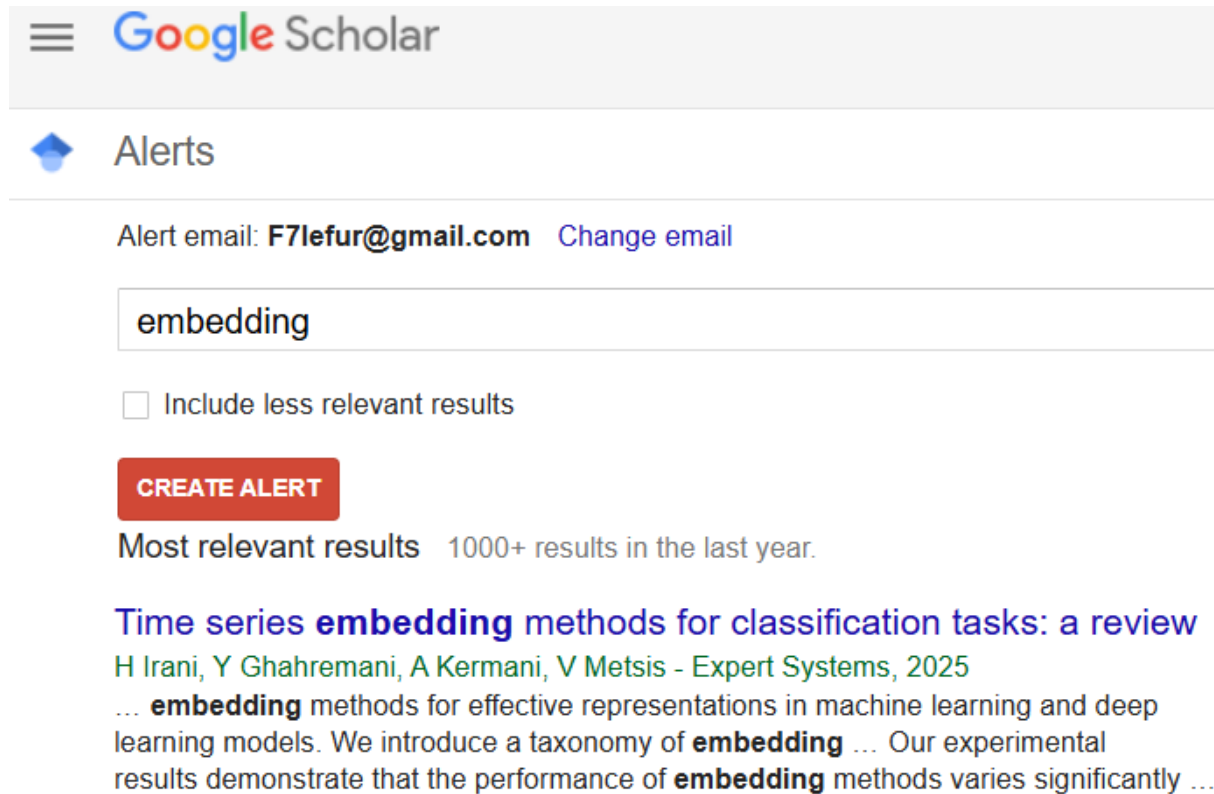
Lien : <https://arxiv.org/pdf/2601.10599v1>

FIGURE 1 – Résultats d'une requête de l'API Arxiv.com

En production, on pourrait automatiser ce genre de requêtes avec une tâche crontab et envoyer un email avec les résultats obtenus.

2.2.2 Les alertes Google Scholar

Le site **Google Scholar** propose un système d'alerte en fonction de mots-clés, nous prévenant par email dès qu'un article correspondant à nos critères vient d'être publié.



Google Scholar

Alerts

Alert email: **F7lefur@gmail.com** [Change email](#)

embedding

☐ Include less relevant results

CREATE ALERT

Most relevant results 1000+ results in the last year.

Time series embedding methods for classification tasks: a review
H Irani, Y Ghahremani, A Kermani, V Metsis - *Expert Systems*, 2025
... **embedding** methods for effective representations in machine learning and deep learning models. We introduce a taxonomy of **embedding** ... Our experimental results demonstrate that the performance of **embedding** methods varies significantly ...

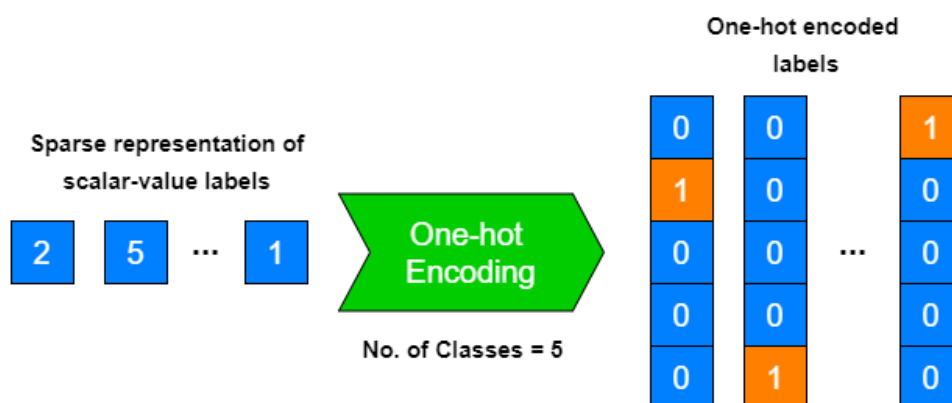
FIGURE 2 – Alertes automatiques avec Google Scholar

3 Représentations Discrètes et Fréquentielles [C6]

Cette première ère de vectorisation se caractérise par l'utilisation de représentations "creuses", où la dimension des vecteurs est proportionnelle à la taille du vocabulaire et où la grande majorité des valeurs vectorielles est nulle.

3.1 Encodage One-Hot

Le One-Hot Encoding est l'approche la plus élémentaire. Chaque mot unique du vocabulaire est encodé en un vecteur binaire, contenant un 1 à la position correspondant à ce mot spécifique et des 0 partout ailleurs.



Inconvénients :

- Les vecteurs sont de dimensions trop grandes (de dimension la taille du dictionnaire),
- Cette méthode ne permet pas au modèle de comprendre la syntaxe du langage,
- Cette méthode ne permet pas de capturer des relations sémantiques entre les mots, c'est-à-dire que 2 mots sémantiquement proches auront des représentations éloignées. (Mathématiquement, le produit scalaire entre deux vecteurs One-Hot différents est toujours nul, ce qui signifie que toutes les paires de mots sont considérées comme orthogonales, on ne peut donc pas tenir compte du calcul de similarité cosinus).

Utilisation : Encoder des variables catégorielles

3.2 Le Modèle Bag-of-Words (BoW)

Le modèle Bag-of-Words (BoW), ou "Sac de Mots," encode des textes en se basant sur l'occurrence des mots.

Fonctionnement :

- Création du vocabulaire à partir des mots uniques d'un ensemble de documents,
- Calcul de l'occurrence de chaque mot du vocabulaire dans l'ensemble des documents,
- Chaque document est représenté par un vecteur où chaque nombre correspond à l'occurrence d'un mot dans le document.

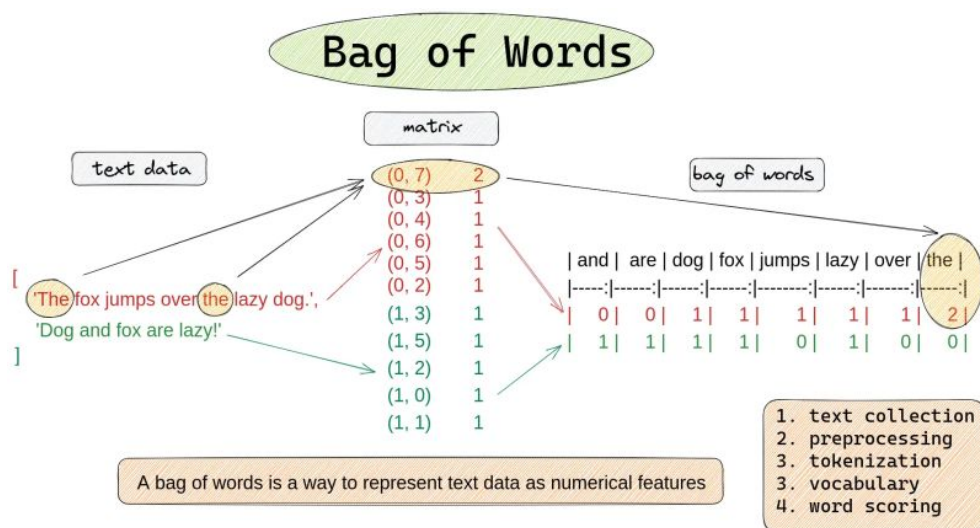


FIGURE 3 – Bag-of-Words

Inconvénients :

- Dimension des vecteurs trop importante,
- Ne comprend pas la syntaxe du langage,
- Ne capture pas la sémantique des mots ou des phrases, elle peut encoder de la même manière des phrases différentes. (Elle ne tient pas compte de l'ordre des mots dans une phrase).
- Il est difficile d'ajouter de nouveaux mots.

Document D1	<i>The child makes the dog happy</i> the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	<i>The dog makes the child happy</i> the: 2, child: 1, makes: 1, dog: 1, happy: 1



	child	dog	happy	makes	the	BoW Vector representations
D1	1	1	1	1	2	[1,1,1,1,2]
D2	1	1	1	1	2	[1,1,1,1,2]

FIGURE 4 – Bag-of-Words

La méthode Bag-of-Words peut être utilisée pour des tâches simples de classification de textes, les calculs sont très rapides.

3.3 TF-IDF (Term Frequency-Inverse Document Frequency)

La méthode TF-IDF améliore le modèle Bag of Words en transformant le simple "comptage" de mots en une mesure de l'importance relative de chaque mot au sein d'un document et d'un corpus.

La méthode combine deux facteurs :

- Term Frequency (TF) : La fréquence d'apparition d'un terme dans un document.
- Inverse Document Frequency (IDF) : L'inverse de la fréquence du document, mesurant à quel point un terme est rare ou commun à travers l'ensemble du corpus.

En notant t un mot et d un document,

$$TF(t, d) = \frac{\text{Nombre de fois où le mot } t \text{ apparaît dans le document } d}{\text{Nombre de termes dans le document } d}$$

$$IDF(t) = \log \left(\frac{\text{Nombre total de documents}}{\text{Nombre total de documents où le terme } t \text{ apparaît}} \right)$$

Le score TF-IDF est le produit de ces deux composantes. Ce mécanisme réduit l'importance des mots trop courants (par exemple les stop words) qui apparaissent dans presque tous les documents.

$$TFIDF(t, d) = TF(t, d) * IDF(t)$$

Remarques :

- 1 BoW apporte trop d'importance aux mots trop courants (par exemple les stop words), le terme IDF permet de résoudre ce problème (on parle de réduction du bruit). A l'inverse un mot rare dans le corpus de documents mais présent dans un document reçoit un score plus élevé (meilleure valorisation des mots discriminants). En conséquences les algorithmes de Machine Learning seront plus efficaces.
- 2 La vectorisation TF-IDF est uniquement basée sur des calculs de fréquence et ne tient compte ni de la sémantique des mots ni du contexte des mots. Il n'y a donc pas d'arithmétique possible sur les vecteurs TF-IDF (comme par exemple : roi - homme + femme = reine)

3.4 Limitations de ces méthodes

Toutes ces méthodes discrètes basées sur la fréquence des mots présentent deux inconvénients majeurs :

- La haute dimensionnalité des vecteurs creux freinent leur utilisation dans le contexte du Deep Learning (on cherche toujours à réduire la dimension des données).
- Elles ne permettent pas de capturer la structure grammaticale du langage ni de capturer la sémantique des mots. Par exemple, statistiquement, les mots "voiture" et "automobile" sont traités comme deux entités distinctes bien que leur signification soit quasi identique.

4 Les Plongements Lexicaux Statiques et Prédicatifs (Word Embeddings) [C6]

Dans ces méthodes, les représentations vectorielles seront "denses", de faibles dimensions et avec des valeurs continues.

4.1 Word2Vec : Architectures Neuronales Légères

Word2Vec repose sur l'idée que les mots apparaissant dans des contextes similaires ont des significations proches et que leurs vecteurs doivent être géométriquement proches dans l'espace vectoriel, ce qui capturerait la similarité sémantique et syntaxique des textes. Word2Vec consiste en 2 architectures principales **CBOW (Continuous Bag of Words)** et **Skip-gram** qui utilisent des réseaux de neurones peu profonds (deux couches) pour apprendre des vecteurs en reconstruisant les contextes linguistiques des mots.

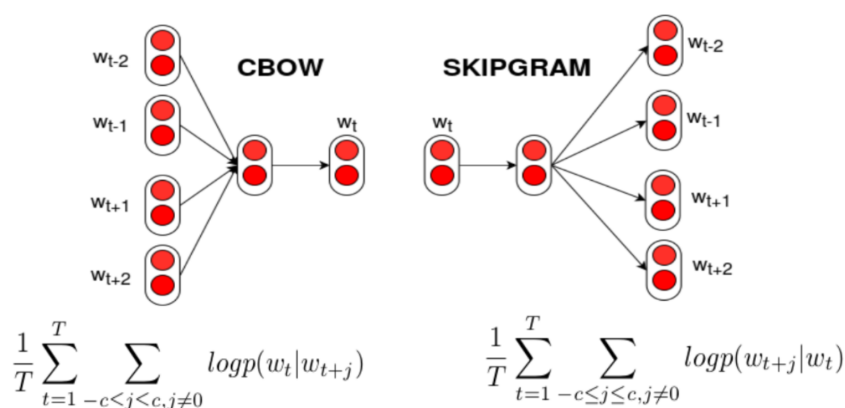


FIGURE 5 – Architectures CBOW et Skip-gram

Dans l'approche CBOW, la tâche consiste à prédire le mot central à partir de la somme vectorielle de ses mots voisins dans une fenêtre de contexte donnée. (tâche de remplissage du blanc)

Le modèle Skip-gram inverse la tâche de CBOW : il utilise le mot central pour prédire les mots qui l'entourent (son contexte).

Remarques

- CBOW est généralement plus rapide à entraîner que Skip-gram,
- CBOW est plus efficace pour les mots fréquentes,
- Skip-gram est plus efficace pour obtenir des représentations de qualité pour les mots peu fréquents,
- Word2Vec capture la sémantique des mots (les représentations vectorielles de **chat** et **chaton** vont être proches au sens de la similarité cosinus),
- Word2Vec capture la syntaxe du langage (compréhension du pluriel, du genre, superlatif, contraire etc ...),
- Word2Vec permet de construire une arithmétique sur les mots par exemple : "Le roi - homme + femme = la reine".

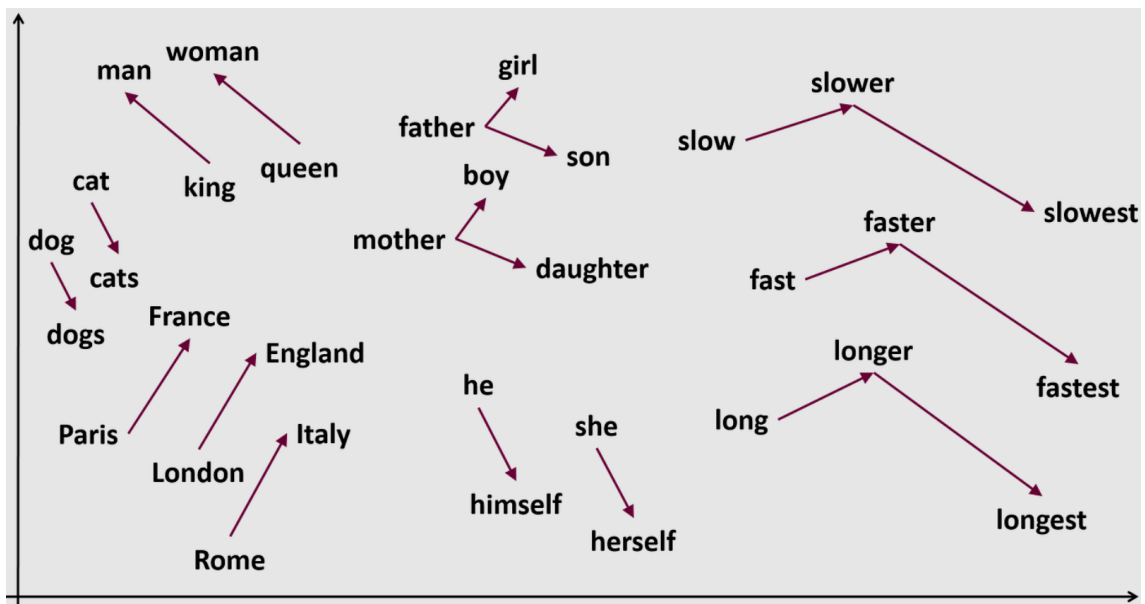


FIGURE 6 – Géométrie avec Word2Vec

4.2 GloVe (Global Vector)

GloVe (Global Vectors) est une méthode de vectorisation utilisant l'apprentissage non supervisé et combinant les approches globales et locales en s'appuyant sur une matrice de co-occurrence construite à partir de l'ensemble du corpus de document.

1ère étape :

GloVe construit d'abord une matrice de co-occurrence mot-mot, c'est-à-dire que chaque élément indique combien de fois une paire de mots apparaît à proximité l'une de l'autre dans une fenêtre de contexte définie.

Ensuite GloVe part d'une observation simple : Le rapport des probabilités de co-occurrence peut révéler des relations sémantiques.

Par exemple :

$P(\text{solide} \mid \text{glace}) / P(\text{solide} \mid \text{vapeur})$ est élevé.

$P(\text{gazeux} \mid \text{glace}) / P(\text{gazeux} \mid \text{vapeur})$ est faible.

Ces rapports permettent de distinguer des concepts même si les fréquences absolues sont différentes.

2ème étape : l'apprentissage

L'idée : on veut que la différence entre les vecteurs de mots encode ces rapports de probabilités.

L'apprentissage consiste alors à factoriser cette matrice de manière à ce que le produit scalaire entre un vecteur de mot et un vecteur de contexte (également appris) s'approche du logarithme de leur nombre de co-occurrences. Une optimisation par moindres carrés pondérés permet de minimiser l'écart entre cette approximation et les valeurs observées, en accordant moins d'importance aux co-occurrences très rares ou très fréquentes. Le résultat final est un espace vectoriel où la distance et la direction entre vecteurs reflètent à la fois des similarités sémantiques et syntaxiques, permettant par exemple d'effectuer des opérations analogiques comme « roi – homme + femme = reine ».

Avantage : Contrairement à Word2Vec (local), GloVe utilise les statistiques global (il voit tout le corpus).

Limites

- Statique : un mot a toujours la même représentation, même dans des contextes différents.
- Ne gère pas les mots hors vocabulaire (OOV).

Source : <https://www.geeksforgeeks.org/nlp/glove-word-embedding-in-nlp/>

4.3 FastText : Robustesse par les Sous-Mots (Subword)

Le problème des méthodes comme Word2Vec et GloVe est leur incapacité à générer des vecteurs pour les mots hors-vocabulaire, car ils attribuent un vecteur unique **à chaque mot entier vu pendant l'entraînement**. **Objectif de FastText** : Apprendre des représentations vectorielles denses des mots qui Capturent la sémantique (comme Word2Vec ou GloVe) en utilisant une approche basée sur les sous-mots (ou n -grammes). Si un mot hors-vocabulaire est rencontré, FastText génère son plongement en sommant les vecteurs des n -grammes de caractères qui le composent.



FIGURE 7 – Somme des n -grammes avec FastText

Avantages

- Gestion OOV (Out of Vocabulary) : Même un mot entièrement nouveau peut être encodé de manière significative si ses fragments (ses n -grammes) ont été vus pendant l'entraînement.
- FastText est particulièrement robuste pour les langues riches en morphologie (où les mots changent beaucoup par flexion ou conjugaison) et pour les fautes de frappe ou les mots rares, car il exploite l'information au niveau des caractères.
- FastText est léger et rapide en inférence.

Limites

- Taille du modèle plus grande : il faut stocker des vecteurs pour tous les n -grammes (ex. centaines de milliers).
- Moins efficace pour les mots très courts (ex. "a", "l") : peu de sous-mots utiles.
- C'est également un embedding statique : pas de contexte dynamique.
- Sensible au bruit orthographique : fautes de frappe peuvent générer des n -grammes inconnus.

4.4 La Limite de la Sémantique Statique

Malgré les avancées de Word2Vec, GloVe et FastText en termes de similarité sémantique, ils partagent un défaut fondamental : ils sont statiques. Chaque mot se voit attribuer un vecteur unique, ignorant le contexte spécifique de son apparition.

Cette contrainte empêche de distinguer des représentations pour le mot "avocat" selon que l'on parle du fruit ou de la fonction. Les modèles statiques ne peuvent représenter que la "sémantique moyenne" du mot dans le corpus. La nécessité de générer un vecteur dynamique pour chaque instance d'un mot, en fonction de son environnement, a conduit à l'avènement des modèles contextuels basés sur les Transformers.

5 L'Encodage Contextuel et des Transformers [C6]

Les méthodes vues précédemment ne considèrent pas les mots dans leur contexte, par la suite des méthodes basées sur les réseaux récurrents (par exemple **ELMo (embeddings from Language Model)**) ont considéré les textes de façon séquentielles, c'est grâce à cela que le problème de **l'avocat** (le fruit ou le métier) a été résolu pour la première fois. Cependant les RNN et les LSTM sont très lents à entraîner, les LSTM ont du mal à garder en mémoire une information située au tout début d'un texte très long et les phrases doivent être découpées pour être mis dans des vecteurs de taille fixe ce qui crée une perte d'information contextuelle.

5.1 L'Architecture Transformer : le mécanisme d'Attention

Le **Transformer** a marqué une révolution en s'affranchissant des réseaux neuronaux récurrents, en effet ici un mot n'est pas seulement rattaché aux mots qu'il suit mais va être rattaché à tous les autres mots, ensuite Le cœur de l'architecture **Transformer** est le **mécanisme d'attention** :

- 1 Pour chaque mot (ou token), le modèle calcule un score d'importance par rapport à tous les autres mots de la phrase.
- 2 Il utilise ensuite ces scores pour pondérer l'information : les mots les plus pertinents contribuent davantage à la représentation finale, tandis que les mots moins utiles sont « atténués ».

Ainsi les représentations dans un Transformer évoluent à travers les couches du modèle pour intégrer le contexte global de la phrase et finalement un même mot peut générer des vecteurs différents selon le contexte du texte.

- Le transformer résout le problème de **polysémie** (un mot peut avoir plusieurs sens selon son contexte),
- l'architecture transformer peut se paralléliser (contrairement aux RNN ou LSTM) donc des temps d'entraînement réduits.
- Un regard longue distance : le modèle transformer capture les liens entre les mots quelque soit leur distance dans un texte.

5.2 BERT (Bidirectional Encoder Representations from Transformers)

Développé par Google en 2018, BERT est un modèle utilisant l'encodeur du Transformer de façon **bidirectionnelle** : il analyse le contexte d'un mot en regardant simultanément ce qui précède (contexte gauche) et ce qui suit (contexte droit).

La bidirectionnalité confère à BERT une capacité de compréhension contextuelle complète pour chaque mot. Contrairement aux modèles statiques, BERT génère un vecteur unique et dynamique pour chaque apparition du mot dans une phrase donnée. Par conséquent, il peut attribuer des vecteurs différents à même mot selon son contexte, résolvant ainsi lui aussi la **polysémie**.

BERT est pré-entraîné sur deux tâches principales pour maximiser sa capacité de compréhension :

- 1 Masked Language Model (MLM) : Le modèle doit prédire des mots masqués dans la phrase à partir du contexte environnant.
- 2 Next Sentence Prediction (NSP) : Le modèle doit prédire si une phrase B suit logiquement une phrase A. Cela permet au modèle de comprendre les dépendances et les relations qui existent entre des phrases consécutives.

Grâce à ces innovations, BERT a rapidement établi des performances de pointe sur de nombreux benchmarks de compréhension. Des variantes ont été développées pour optimiser les performances ou les ressources, telles que **DistilBERT** (version allégée, 40% plus rapide pour 97% des performances) et **CamemBERT** (modèle spécifiquement entraîné pour la langue française).

5.3 GPT (Generative Pre-trained Transformer)

Les modèles de la série **GPT**, développés par **OpenAI**, sont des modèles génératifs basés sur les Transformers. Leur fonctionnement est intrinsèquement unidirectionnel : ils regardent uniquement le contexte précédent (gauche) pour prédire le mot suivant.

Cette structure unidirectionnelle les rend excellents pour la génération de texte cohérent, créatif et dans des styles variés, mais moins bons dans la compréhension que les modèles bidirectionnels comme BERT.

5.4 Choix du modèle : Encodage vs. Génération

L'encodeur bidirectionnel (BERT) est optimisé pour les tâches discriminatives (analyse, classification, réponse aux questions), car il nécessite une vue complète de la séquence pour déterminer le contexte d'un mot. En revanche, le décodeur unidirectionnel (GPT) est optimisé pour la génération séquentielle, imitant la manière dont le langage est naturellement produit.

Cette divergence est cruciale pour bien choisir son modèle en fonction des cas d'usages : **BERT** est la référence pour l'analyse fine (classification, NER) alors que **GPT** est le moteur des interfaces conversationnelles et de la création de contenu.

Pour la tâche de **similarité sémantique** qui nous intéresse dans ce projet nous étudierons donc attentivement les modèles de type **BERT**

6 Services d'intelligence artificielle pour de l'embedding [C7, C8]

Parmi les services d'intelligence artificielle permettant de faire de l'embedding de textes, on retrouve l'API de **Mistral AI** (avec ses modèles comme 'mistral-embed'), **Google Cloud Vertex AI** ('text-embedding-004'), ainsi que la plateforme **Hugging Face** qui offre à la fois une Inference API pour des modèles pré-entraînés et un accès direct à des modèles open source comme ceux de la bibliothèque Sentence Transformers (par exemple 'all-MiniLM-L6-v2' ou 'bge' series).

A noter que certains de ces services sont payants ou en accès limités, citons par exemple :

- OpenAi (payant)
- Google Cloud Vertex AI (accès limité)
- Mistral (accès limité)

Nous allons voir comment intégrer et paramétrer des modèles d'embedding dans notre projet.

6.1 Mistral

Nous utilisons l'API Mistral pour faire de l'embedding. Il nous suffit de créer un compte sur le site Mistral.ai, de générer une clé API, nous pouvons ensuite requêter l'API Mistral et faire de l'embedding de textes.

```
1 import os
2 from mistralai import Mistral
3 from dotenv import load_dotenv
4
5 load_dotenv()
6 client = Mistral(api_key=os.getenv('MISTRAL_API_KEY'))
7
8 texts = ["Bonjour, ceci est un exemple de texte", "Un deuxieme texte a
9         encoder"]
10
11 response = client.embeddings.create(
12     model="mistral-embed",
13     inputs=texts
14 )
15
16 embeddings = [embedding.embedding for embedding in response.data]
17 print(f"Nombre d'embeddings : {len(embeddings)}")
18 print(f"Dimension d'un embedding : {len(embeddings[0])}")
```

```
Nombre d'embeddings : 2
Dimension d'un embedding : 1024
```

6.2 Cohere

```
1 import os
2 import cohere
3 from data import SENTENCES
4 from benchmark_utils import benchmark
5 from dotenv import load_dotenv
6 import numpy as np
7
```

```

8 load_dotenv()
9
10 cohere_api_key = os.getenv('COHERE_API_KEY')
11 co = cohere.Client(cohere_api_key)
12
13 texts = ["Bonjour, ceci est un texte en francais",
14          "Hello, this is an English text",
15          "Hola, este es un texto en espanol"]
16
17 response = co.embed(
18     texts=texts,
19     model="embed-multilingual-v3.0",
20     input_type="search_document"      # Type d'utilisation
21 )
22
23 embeddings = response.embeddings
24 print(f"Nombre de textes : {len(embeddings)}")
25 print(f"Dimension d'un embedding : {len(embeddings[0])}")
26 print(f"Premier embedding (10 premieres valeurs) : {embeddings[0][:10]}")
27
28 # Conversion en numpy array pour manipulation
29 embeddings_array = np.array(embeddings)
30 print(f"\nShape du tableau d'embeddings : {embeddings_array.shape}")

```

Type : API cloud premium

Modèles : embed-multilingual-v3.0, embed-english-v3.0

Tarifs : 0.10–0.40 \$/million tokens

Spécialité : Embeddings optimisés pour la recherche

6.3 Hugging Face

```

1 import requests
2
3 API_URL = "https://api-inference.huggingface.co/pipeline/feature-extraction/
4 sentence-transformers/all-MiniLM-L6-v2"
5 headers = {"Authorization": f"Bearer
6 hf_LEoeJZgEakbqPhXXXXXXXXXXQQNIhNUTbDlYPZIXZ"}
7
8 def get_embeddings(texts):
9     response = requests.post(API_URL, headers=headers, json={"inputs": texts
10 })
11     return response.json()
12
13 texts = ["Bonjour, ceci est un texte", "Hello, this is a text"]
14 embeddings = get_embeddings(texts)
15 print(f"Nombre d'embeddings : {len(embeddings)}")

```

Type : Bibliothèque Python locale

Coût : 100% gratuit

Modèles : Tous modèles publics

Installation : pip install sentence-transformers

Inconvénients :

Requiert du CPU/GPU local

Installation initiale

Moins performant que les modèles propriétaires.

6.4 OpenRouter

[OpenRouter.ai](https://openrouter.ai) est un agrégateur d'APIs LLM — il donne accès à plusieurs modèles (dont certains gratuits) par une seule interface, on peut ainsi comparer facilement différents services et différentes tâches liées au LLM en ne gérant qu'une seule clé API.

Exemple d'utilisation de **OpenRouter** avec le modèle **thenlper/gte-base**"

```
1 import os
2 import requests
3 import json
4 from dotenv import load_dotenv
5
6 load_dotenv()
7 openrouter_api_key = os.getenv('OPENROUTER_API_KEY')
8
9 response = requests.post(
10     url="https://openrouter.ai/api/v1/embeddings",
11     headers={
12         "Authorization": f"Bearer {openrouter_api_key}",
13         "Content-Type": "application/json",
14     },
15     data=json.dumps({
16         "model": "thenlper/gte-base",
17         "input": "Your text string goes here",
18         "encoding_format": "float"
19     })
20 )
21
22 print(f"Longueur de l'embedding: {len(response.json()['data'][0]['embedding'])}")
```

Quelques modèles disponibles d'embedding sur OpenRouter :

- thenlper/gte-base, thenlper/gte-large
- intfloat/e5-large-v2, intfloat/e5-base-v2, intfloat/multilingual-e5-large
- sentence-transformers/paraphrase-minilm-l6-v2, sentence-transformers/all-minilm-l12-v2, sentence-transformers/multi-qa-mpnet-base-dot-v1, sentence-transformers/all-mpnet-base-v2, sentence-transformers/all-minilm-l6-v2
- baai/bge-base-en-v1.5, baai/bge-large-en-v1.5, baai/bge-m3
- mistralai/mistral-embed-2312, mistralai/codestral-embed-2505
- google/gemini-embedding-001
- openai/text-embedding-ada-002, openai/text-embedding-3-large openai/text-embedding-3-small
- qwen/qwen3-embedding-8b, qwen/qwen3-embedding-4b, qwen/qwen3-embedding-0.6b

7 Évaluation des modèles d'embeddings [C7]

Maintenant que nous avons identifié des services d'IA proposant des modèles d'embedding nous pouvons faire de l'embedding sur des textes et comparer ces embeddings.

7.1 Evaluation à l'aide de MTEB

Comment comparer des embeddings ? Comment évaluer un modèle d'embedding ?

Nous allons utiliser [MTEB](#) (Massive Text Embedding Benchmark), c'est un framework d'évaluation standardisé de modèles d'embedding qui regroupe 56 datasets répartis en 8 catégories de tâches de NLP. Développé par la communauté, il est devenu la référence pour évaluer les capacités générales des modèles d'embedding, cependant il présente une limite : il est principalement concentré sur l'anglais.

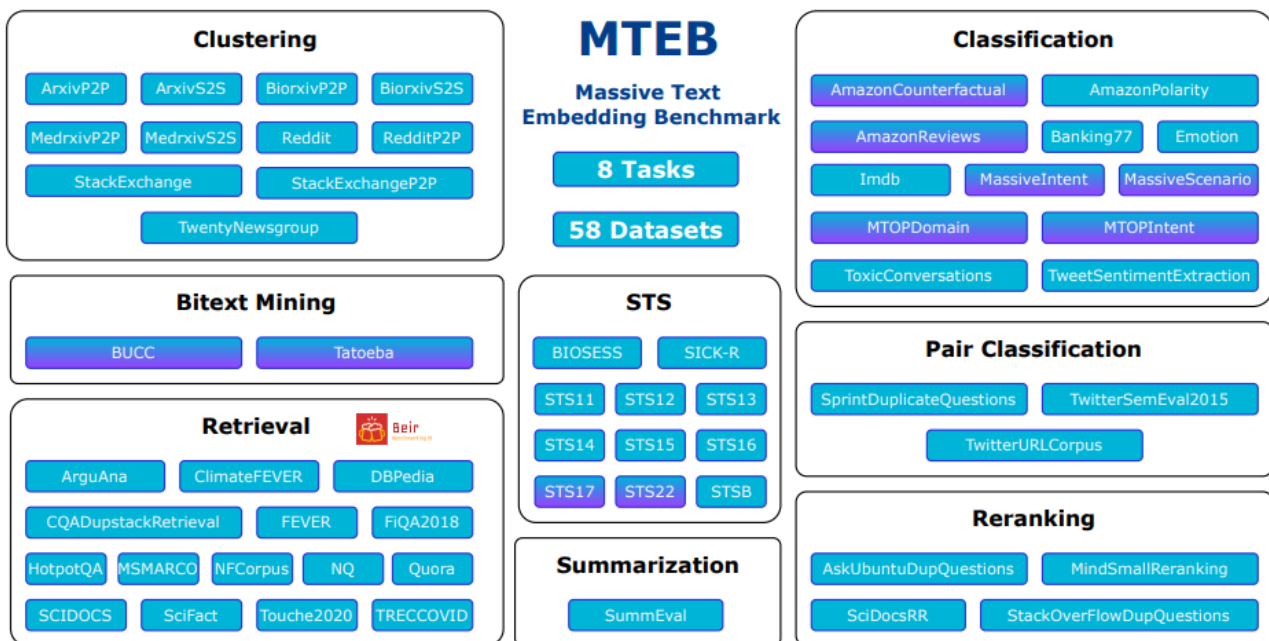


FIGURE 8 – Tâches et Dataset MTEB

7.1.1 Les 8 Catégories de Tâches de MTEB

Ce sont les tâches fondamentales du NLP.

- 1 Similarité Sémantique (Semantic Similarity)** Objectif : Mesurer à quel point deux phrases sont sémantiquement similaires
Datasets clés :
STS 2012-2016 : Paires de phrases annotées avec des scores de similarité (0-5)
STS Benchmark : Version standardisée avec splits train/dev/test
Métrique principale : Corrélation de Spearman entre similarités prédites et scores réels
- 2 Rétreival (Information Retrieval)** Objectif : Retrouver des documents pertinents à partir d'une requête
Datasets : MS MARCO : Requêtes réelles de Bing avec passages pertinents
- 3 Clustering** Objectif : Grouper des documents similaires sans supervision
- 4 Classification** Objectif : Prédire la catégorie d'un texte avec supervision

- 5 **Appariement de Paires (Pair Classification)** Objectif : Déterminer si deux textes ont une relation spécifique
- 6 **Reranking** Objectif : Réordonner des résultats de recherche par pertinence
- 7 **Bittext mining** Objectif : Trouver des paires de phrases qui sont des traductions l'une de l'autre à partir de deux ensembles de textes dans des langues différentes.
- 8 **Summarization** Objectif : Évaluer la capacité à suivre des instructions pour le résumé

Nous utiliserons les modèles disponibles sur la plateforme **MTEB**, elle dispose d'un grand nombre de modèles : [Modèles MTEB](#)

7.1.2 Evaluation des modèles sur la tâche de similarité sémantique

Il est bien trop gourmand en ressources d'évaluer nos modèles sur les dataset complets de MTEB sur ces 8 tâches de NLP. Nous ne ferons l'évaluation que sur la tâche de **Similarité sémantique (STS : Semantic Textual Similarity)** car c'est la tâche qui nous intéresse dans le projet de **moteur de recherche** du **Crédit Agricole**.

Le Dataset **STSBenchmark** contient 8,628 paires de phrases, réparties en :

- Train : 5,749 paires
- Dev : 1,500 paires
- Test : 1,379 paires

Chaque observation du dataset consiste en une paire de phrase et un score de similarité (entre 0 et 5) établi manuellement par une personne.

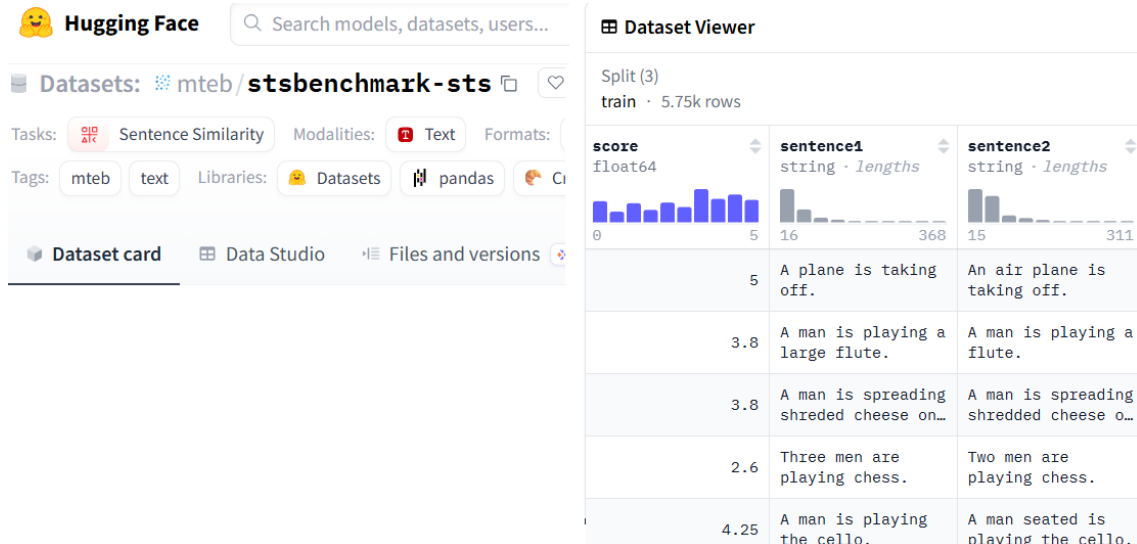


FIGURE 9 – STS Benchmark - Huggingface

Ensuite la librairie MTEB python permet d'évaluer les modèles.

```

1 # Modele
2 model = mteb.get_model("BAAI/bge-small-en-v1.5")
3 # Tache
4 tasks = mteb.get_tasks(tasks=['STSBenchmark'])
5 # evaluate
6 results = mteb.evaluate(model, tasks=tasks)
7 results.to_dataframe(format="long")

```

	model_name	task_name	score
0	BAAI/bge-small-en-v1.5	STSBenchmark	0.85863

FIGURE 10 – Score de corrélation de Spearman du modèle BGE

Par défaut le score affiché est le coefficient de **Spearman**, on peut s'en assurer en affichant tous les scores calculés lors de l'évaluation.

```
results.task_results[0].scores
✓ 0.0s
{'test': [{'pearson': 0.843129,
'spearman': 0.85863,
'cosine_pearson': 0.843129,
'cosine_spearman': 0.85863,
'manhattan_pearson': 0.851387,
'manhattan_spearman': 0.856401,
'euclidean_pearson': 0.853616,
'euclidean_spearman': 0.85863,
'hf_subset': 'default',
'languages': ['eng-Latn']}]}
```

FIGURE 11 – Tous les scores du modèle BGE

Nous pouvons désormais évaluer plusieurs modèles sur différents Datasets.

```
1 df = pd.DataFrame()
2
3 for model_name in [
4     "sentence-transformers/all-MiniLM-L6-v2",
5     "BAAI/bge-small-en-v1.5",
6     "Cohere/Cohere-embed-multilingual-light-v3.0",
7     "Qwen/Qwen3-Embedding-0.6B",
8 ]:
9
10     model = mteb.get_model(model_name)
11
12     tasks = mteb.get_tasks(tasks=[
13         'STS12',
14         'STS13',
15         'STS14',
16         'STS15',
17         'STS16',
18         'STSBenchmark',
19         'SICK-R'
20     ])
21
22     # evaluate
23     results = mteb.evaluate(model, tasks=tasks)
24     df = pd.concat([df, results.to_dataframe(format="long")])
```

Nous résumons les résultats des modèles évalués dans un DataFrame.

	model_name	task_name	score
0	sentence-transformers/all-MiniLM-L6-v2	SICK-R	0.775824
1	sentence-transformers/all-MiniLM-L6-v2	STS12	0.723690
2	sentence-transformers/all-MiniLM-L6-v2	STS13	0.806032
3	sentence-transformers/all-MiniLM-L6-v2	STS14	0.755891
4	sentence-transformers/all-MiniLM-L6-v2	STS15	0.853896
5	sentence-transformers/all-MiniLM-L6-v2	STS16	0.789895
6	sentence-transformers/all-MiniLM-L6-v2	STSBenchmark	0.820325
0	BAAI/bge-small-en-v1.5	SICK-R	0.794099
1	BAAI/bge-small-en-v1.5	STS12	0.774434
2	BAAI/bge-small-en-v1.5	STS13	0.829800
3	BAAI/bge-small-en-v1.5	STS14	0.818354
4	BAAI/bge-small-en-v1.5	STS15	0.872654
5	BAAI/bge-small-en-v1.5	STS16	0.849340
6	BAAI/bge-small-en-v1.5	STSBenchmark	0.858630
0	Cohere/Cohere-embed-multilingual-light-v3.0	SICK-R	0.777348
1	Cohere/Cohere-embed-multilingual-light-v3.0	STS12	0.756016
2	Cohere/Cohere-embed-multilingual-light-v3.0	STS13	0.762409
3	Cohere/Cohere-embed-multilingual-light-v3.0	STS14	0.773960
4	Cohere/Cohere-embed-multilingual-light-v3.0	STS15	0.861588
5	Cohere/Cohere-embed-multilingual-light-v3.0	STS16	0.839054
6	Cohere/Cohere-embed-multilingual-light-v3.0	STSBenchmark	0.834210
0	Qwen/Qwen3-Embedding-0.6B	SICK-R	0.848571
1	Qwen/Qwen3-Embedding-0.6B	STS12	0.833912
2	Qwen/Qwen3-Embedding-0.6B	STS13	0.915034
3	Qwen/Qwen3-Embedding-0.6B	STS14	0.868411
4	Qwen/Qwen3-Embedding-0.6B	STS15	0.913529
5	Qwen/Qwen3-Embedding-0.6B	STS16	0.903521
6	Qwen/Qwen3-Embedding-0.6B	STSBenchmark	0.912247

FIGURE 12 – Evaluation STS all-MiniLM-L6-v2

Remarques

- Dans l'évaluation de nos modèles (versions light pour certains) sur les tâches de STS (Semantic Textual Similarity), le meilleur modèle est **Qwen3-Embedding-0.6B**.
- A noter également les bonnes performances du modèles **all-miniLM-L6-v2** au vu de son poids (seulement 33 millions de paramètres).

```

1 results.task_results[0].scores
2 'pearson': 0.827406,
3 'spearman': 0.820325,
4 'cosine_pearson': 0.827406,
5 'cosine_spearman': 0.820325,
6 'manhattan_pearson': 0.824914,
7 'manhattan_spearman': 0.819455,

```

7.2 Leaderboard MTEB

La plateforme MTEB propose des tableaux de benchmak où l'on peut choisir ses modèles en fonction des scores sur les différentes tâches.

Model	SICK-R	STS12	STS13	STS14	STS15	STS17	STS22.v2	STSB	STSBenchmark	STSES	SemRe124STS
Octen-Embedding-8B	88.16	86.24	94.00	90.52	93.82	91.79	73.95	86.30	93.61	74.81	64.37
Qwen3-Embedding-8B	88.60	86.14	94.22	90.42	93.73	92.12	72.59	85.50	93.60	74.08	64.43
llama-embed-nemotron-8b	85.37	83.36	91.96	87.84	91.83	89.96	71.54	86.14	90.55	80.34	70.28
Qwen3-Embedding-4B	88.06	86.61	94.39	90.94	93.77	91.78	72.95	86.10	93.70	72.75	63.17
gemini-embedding-001	82.75	81.55	89.89	85.41	90.44	88.58	71.69	85.50	89.08	81.75	73.14
bilingual-embedding-large	81.01	85.52	89.37	91.61	92.02	88.71	71.35	84.93	89.28	80.32	67.58
KaLM-Embedding-Gemma3-12B-25	82.46	80.92	89.83	85.91	90.13	87.31	70.25	84.49	89.00	79.41	71.05
Qwen3-Embedding-0.6B	84.83	83.00	91.76	87.10	91.45	85.50	71.83	84.62	91.13	76.95	59.37
Seed1.6-embedding-1215	85.34	82.93	90.77	90.11	91.40	81.60	69.00	84.09	89.40	78.70	58.24
Cohere-embed-multilingual-v3	82.18	77.62	85.16	80.02	88.92	90.09	69.36	81.42	88.79	78.76	63.84
gte-Qwen2-1.5B-instruct	80.49	77.25	87.79	82.91	88.45	90.33	69.40	79.40	86.38	76.52	61.53
bge-m3-custom-fr	77.63	81.36	81.55	80.06	88.02	80.25	68.89	82.76	84.90	77.98	66.39
gte-multilingual-base	79.34	77.51	85.55	81.68	88.94	82.59	68.79	82.70	86.46	70.77	65.23
LLM2Vec-Mistral-7B-Instruct	83.70	78.80	86.37	84.04	88.99	90.19	62.03	69.50	88.65	74.51	51.76
text-embedding-3-large	79.00	72.84	86.10	81.15	88.49	90.22	69.29	80.25	83.56	74.20	65.25
voyage-large-2-instruct	81.92	73.34	84.96	82.67	87.64	74.89	70.30	80.55	87.59	78.82	56.07
gte-Qwen1.5-7B-instruct	79.15	76.52	88.63	83.32	87.50	87.79	68.80	81.37	87.35	72.51	55.28
bge-m3	79.72	78.73	79.60	79.00	87.81	79.65	70.03	81.90	84.87	77.50	65.38
all-MiniLM-L12-v2	79.32	73.08	82.13	76.73	85.58	36.28	36.73	41.90	83.09	72.67	37.93
all-MiniLM-L6-v2	77.58	72.37	80.60	75.59	85.39	36.44	41.79	42.05	82.03	64.33	41.58
sentence_croissant_alpha_v0.	64.88	65.55	78.91	71.50	79.90	58.71	59.91	46.96	77.15	78.34	43.54

FIGURE 13 – Leaderboard MTEB - STSBenchmark

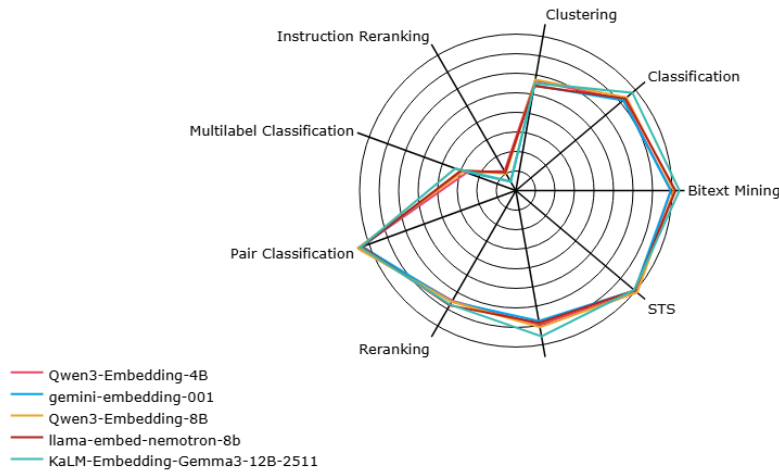


FIGURE 14 – MTEB - Radar plot

8 Conclusion

Ce travail a permis de mettre en place une démarche structurée de veille technologique autour des méthodes de vectorisation de textes, en particulier à des fins de similarité sémantique pour un moteur de recherche.

Les ressources théoriques combinent de publications et solutions récentes mais aussi la consultation de sources plus anciennes permettant ainsi de retracer l’histoire des méthodes de vectorisation.

L’étude des différentes architectures de méthodes de vectorisation a permis d’en comprendre les principes de fonctionnement et de centrer nos choix de modèles en fonction de la tâche à réaliser. La réalisation de benchmarks sur plusieurs modèles, notamment à travers la tâche de similarité sémantique, a fourni une base de comparaison des performances.

Recommandations Finales pour notre choix de modèle :

- On choisira un modèle basé sur les **transformer** pour une meilleure capture de la sémantique des mots, ce sont désormais les références pour la plupart des tâches de NLP,
- On choisira un modèle de type **BERT**, meilleur que **GPT** pour des tâches de classification ou de similarité sémantique,
- Le modèle **Sentence-CamemBERT-large** est conçu pour le français,
- Pour une tâche de similarité sémantique, le DataSet **STSB** est parfaitement adapté,
- Il faut considérer le compromis précision-vitesse dans le choix d’un modèle.

8.1 Tendances Futures

L’avenir de l’encodage vectoriel se dirige vers la **multimodalité**, des modèles pionniers comme **CLIP (Contrastive Language-Image Pre-training)** utilisent l’apprentissage contrastif pour projeter textes, images et sons dans un même espace vectoriel sémantique, facilitant ainsi les applications cross-modales.

Parallèlement, les embeddings contextuels évoluent vers des modèles bien plus légers, rendant leur déploiement embarqué ou à grande échelle économiquement viable.

Enfin, d’autres méthodes permettront de capturer des nuances sémantiques complexes comme l’ironie, le raisonnement ou les relations temporelles.